Agent Memory 产品落地 · 外部对照选型调研报告

调研日期：2026-09-28 | **结果数**：30（学术 A 14 · 工业 I 10 · 开源 O 6） | **检索范围**：以 2024-09 至 2026-09 为主（个别奠基工作例外）；中英文；渠道为 `web_search`、arXiv API、官方文档直取、`api.github.com`（**无法 `git clone`**） **对照基准**：本项目「符号化案例记忆 + 轻量关系层（零向量库、零图库）」——确定性加权打分（关键词×3 / MITRE×2 / 指标×2 / 签名×1 / 正则探针×4）+ 结构化案例库（22 案例）+ 人工确认的一跳关系边

目录

摘要 1
1 调研问题与方法 2
1.1 课题界定 2
1.2 检索与筛选过程 3
1.3 评价标准 3
2 学术线索结果 3
2.1 检索机制：向量层到底什么时候才必要 4
2.2 关系层：从全图到三关系一跳 4
2.3 写入、代价与风险：三个被低估的约束 5
2.4 评测方法与基准：口径不可比是系统性问题 8
3 工业线索结果 9
3.1 零向量库的产品先例 9
3.2 图库的触发阈值与自建代价 10
3.3 治理、删除与合规 11
3.4 精度数字的披露与不可比 11
3.5 记忆攻击面与检索计价 13
4 开源线索结果 14
5 交叉对比 17
6 综合评价与观点 18
6.1 证据支持的结论 18
6.2 被高估或过度声称的部分 19
6.3 尚未证明的问题 19
6.4 若现在必须决策，我的建议 20
6.5 建议继续跟踪的方向 20
7 局限 21
附录 A 评分明细 21
附录 B 未纳入但值得注意 22
附录 C 键位对照 24

摘要

- 公开记忆基准的数字不可跨厂商比较，且分歧已经进入公开互诉：同一基准、同一对手系统，分数在一年内依次出现 84% → 75.14% → 58.44%，每一步都伴随方法学指控（[I-07]、[I-08]）；Mem0 与 Zep 各自的 LoCoMo 口径（读模型、判模型、检索预算、context 大小）逐项不同，1.5–2.2 分的差距落在

口径差异量级内（[I-04]）。

- "图"在头部厂商自己的数据里只值约 2%：Mem0 论文自报带图变体总分仅高约 2%，单跳为负增益、多跳零增益，只有时序类受益（[A-14]）；Zep 自述其检索同时持有向量与 BM25 索引（[I-04]）；而"只保留合并/替代/矛盾三类关系 + 一跳扩展、不建全图"的轻量方案在同 backbone 下取得最好成绩且运行时间大降（[A-02]、[A-03]）。
- 向量层是否必要是"工作负载属性"，不是普适结论：同一套检索底座在 LoCoMo 上由 BM25 单通道最强、稠密通道 100% 可跳过，在 LongMemEval 上稠密通道确有增量信息、仅跳过 63% 的查询（[A-01]）。
- 存在明确的反向证据：一个刻意非向量的本地优先架构，在 LongMemEval 公开端点上落后纯 vector-RAG 22.0 / 26.5 个百分点，且作者自己的机制保真审计承认"图结构效用未被证明"（[A-05]）。这是"零向量库"路线必须正面回应的一条。
- "检索到了"与"用对了"必须分开度量：某个公开冲突更新切片上 22 个已报告系统的多跳成绩全部 ≤7%，而把"证据识别"与"策略执行"解耦后单跳达 82%/93%，其中仅替换最后的策略执行器只贡献 2.0 个百分点（[A-04]）。
- 写入与保留策略比检索算法更能决定不可逆损失：在 top-k 检索 + 80k 预算下，可被修正的错误中不可逆占比 0.67–0.73（[A-09]）；且同一论文明确指出"预算—准确率曲线不声明检索机制就不可比较"。
- 投毒是产品化的硬约束：1.2% 的投毒语料把 LongMemEval 准确率从 0.850 打到 0.300，而同一套写时内容筛查对 360 条投毒记忆拒绝 0 条（[A-06]）；工业侧同向——"记忆进系统提示"是已被走通的持久攻击路径（[I-10]），厂商给出的对策是类型化写入、先过滤后排序、记忆不进系统提示（[I-06]）。
- 零向量库不是权宜之计，而是已被交付的产品默认形态：Letta 明写其 MemFS 默认不含语义/向量索引（[I-01]）；Claude Code 的记忆是纯文本文件、按路径寻址，并给出"每个文件目标 200 行以内"的经验上限且承认超限会降低遵循度（[I-02]）；一个 8.5k star 的 Rust 记忆服务把"何时才该加 sqlite-vec"写成可执行触发条件，并以零 LLM 的纯 FTS 在 LongMemEval-S 上取得 hit@1 0.534 / hit@5 0.668（[O-01]）。

总体判断：本次调研支持"现阶段不必上向量库或图库"这一结论，但支持它的理由与项目文档目前的写法不同——**理由不是"符号检索更准"，而是"公开证据显示向量/图的增益在你们这个规模与工作负载下无法被证明，而它们的代价（写入成本、运维面、攻击面、不可比的口径）是可测的"**。反过来说，同一批证据也给出了必须补的三件事：写入侧的生命周期元数据（时间、来源、活跃标志、槽位键）承载了大部分正确率收益（[A-07]），精度治理是"检索+装配"两段问题而非一段（[A-04]），以及"无匹配内容时应拒答"这一能力在公开基准上被系统性回避（[A-13]、[O-01]、[A-14]）——恰好是本项目前瞻测试里 3/3 失败的那一项。我建议的决策是：**维持零向量/零图库的架构判断，但把下两个迭代的资源从"要不要加图"转到"写入侧元数据 + 装配分离 + 拒答能力"上**；向量层保持 C6/C7 预留位，并以查询类型分层的测量（而非整体召回）来决定何时启用（[A-01]、[O-01]）。

1 调研问题与方法

1.1 课题界定

- 问题：以本项目「符号化案例记忆 + 轻量关系层（零向量库、零图库）」为对照基准，业界（学术机制、Agent Memory 产品与标准、开源项目）在记忆的写入—检索—关系—评测四环上的做法与边界

分别是什么；哪些外部证据支持或反驳"现阶段不必上向量库/图库，但必须治理检索精度"这一判断。

- 范围：时间窗以 2024-09 至 2026-09 为主（个别奠基工作如 LoCoMo 2024-02 例外）；中英文皆采（实际命中以英文为主）；场景聚焦可长期运行的 Agent 记忆系统，并包含安全检测/日志复用的邻近形态。
- 深度：选型比较型（决策支撑）。每条结果 150–300 字，四段式：做了什么 → 怎么做的 → 结果及其条件 → 意义与局限。
- 排除项：模型自身的长期上下文与位置编码类研究（长上下文注意力、KV cache 压缩）；纯向量数据库的功能与跑分横评；通用 RAG 基础设施评测（含通用 RAG 语料投毒一族）。

1.2 检索与筛选过程

三条线索的检索式概况与漏斗：

阶段	学术	工业	开源	合计
命中候选（去重后）	约 180	约 65	约 20	约 265
通过相关性/可核验门槛	19	21	13	53
最终纳入	14	10	6	30

- 检索量：web_search 共 15 次调用（其中 5 次因上游引擎 429 限流失败，成功 10 次、合计 39 条查询词）；arXiv API 12 次查询（累计返回 246 条）；直接页面取数约 90 次请求；GitHub REST API 约 54 次（未认证配额 60/小时，结束时余 7 次）。
- 停止条件：同一子问题的新措辞不再产出新候选；三个公共基准（LoCoMo / LongMemEval / MemoryAgentBench）的引用网络闭合；工业侧 S6 五个维度在 8 家机构的一手页面上各至少有一条直接表述。
- 条数说明：本次取到区间上限（30 条）。理由有三：用户点名三个必答议题（评测方法与基准、产品化约束、关系/图记忆与实体-动作抽取）各自需要独立证据；记忆领域在 2024-09 之后的产出密度极高（仅 2026 年在 arXiv 上的相关论文即以百计）；且跨厂商基准争议必须有对立双方各自单独成条才能被审计。本文因此仍在上限内，但不再扩充；未纳入但值得注意的材料列于附录 B。

1.3 评价标准

按贡献（0.30）、意义（0.25）、时效（0.20）、置信度（0.25）四维打分，总分 = `0.30×`贡献 `+ 0.25×`意义 `+ 0.20×`时效 `+ 0.25×`置信度，满分 10 分。

时效按本课题自身节奏校准：**记忆系统领域在 2024-09 之后迭代极快（基准、模型栈、厂商口径在 12 个月内多次更换）**，因此 2026 年产出按 10 分、2025 年 8 分、2024 年 6 分、2023 年及更早 4 分起步，再按"结论是否已被后续工作取代"微调。置信度按证据强度给分：同行评审 + 多源印证 8–10；官方一手文档 7.5–8.5（页面无日期者扣分）；高质量预印本且代码公开 5.5–6.5；厂商自评且无第三方复现 4–6；纯营销口径 1–3。

2 学术线索结果

2.1 检索机制：向量层到底什么时候才必要

[A-01] AgentIR: A Workload-Adaptive Cascade Retrieval Substrate for Long-Term Conversational Memory（arXiv:2605.25092，2026-05-24，预印本，评分 7.65）

- 做了什么：把"用哪种融合方式"从固定配置改成每查询决策，并额外决定"这次是否值得跑稠密通道"。
- 怎么做的：在 BM25 / Dense / RRF / agent-aware RRF 之间按查询选择；用置信度触发的级联路由，仅凭 BM25 top-k 的间隔判断是否跳过约 52ms 的稠密通道，跨工作负载重调而不重训；底层是时间分区索引。
- 结果：在 LongMemEval（n=500）上稠密通道确有增量信息，级联跳过 63% 查询、在 LLM 判定准确率持平下快 2.67 倍（按查询类型设阈后 5.76 倍）；在 LoCoMo（n=1,982）上 BM25 单通道本身就是最强单系统，同一触发器自动收敛到 100% 跳过稠密通道（快 132 倍，Hit@5 +0.089）。共享 8 核机器上并发 agent 数由约 154 升至约 1,400。
- 意义与局限：这是本次调研中把"要不要向量"从立场问题变成工作负载测量问题的最直接证据，也说明该按查询类型分层测量而非只看整体召回。局限：作者自建系统、无第三方复现；其检索层指标（Hit@5）与厂商的端到端 LLM 判定数字不可直接比大小。我的评价：这条应当直接覆盖本项目的"补测3"，把它从一次整体测试改造成按查询类型分层的测试设计。

2.2 关系层：从全图到三关系一跳

[A-02] MemoryLACE: Memory Lifecycle-Aware Consolidation and Evidence Retrieval（arXiv:2609.03201，2026-09-02，预印本，评分 7.80）

- 做了什么：论证"只显式建模局部证据生命周期"就足以显著改善长期记忆推理，不需要完整知识图、也不需要全局反思。
- 怎么做的：每条记忆是六元组（独立复述的证据文本、时间元数据、轻量元数据、来源轮次、活跃标志、生命周期关系集）；关系只有三类：merge（重复）、update/supersession（同期状态变化）、contradict（范围内不相容，保持双方活跃）。写入按时间序处理，先语义+同实体取候选、再扩展一跳生命周期邻居，用 cross-encoder 重排决定"哪些进入比较"（重排不决定关系），一次决策只选一个关系-目标对。检索从活跃记忆作锚点做生命周期扩展与关系感知重排。
- 结果：摘要口径为在 BEAM 与 StructMemEval 上同 backbone 比较总体最好，BEAM 端到端运行时间相对最强反思式基线 Hindsight 降低 66.6%；消融显示生命周期扩展与时间感知是主要贡献来源。
- 意义与局限：与本项目"符号化案例 + 人工确认一跳边"几乎同构，是"轻量关系层够用"最直接的学术对应物。局限：关系判定仍由 LLM 决策（粒度漂移问题未被正面处理）；矛盾保持活跃等于写入侧不消解冲突，其评测后果未被单独检验；我（本次）只读到方法与架构章节，66.6% 与"最高"的基线集合未能核；无公开代码。

[A-03] Grounded Continuation: A Linear-Time Runtime Verifier for LLM Conversations（arXiv:2605.14175 v2，2026-09-09，预印本，代码公开，评分 8.20）

- 做了什么：把"关系层"实现为一张带 standing 标签的依赖表，而不是图数据库。
- 怎么做的：LLM 只做一件事——把每句话分类为八种认识论操作（Observe/Hypothesize/Support/Undermine/Revise/Expand-Awareness/Resolve/Question）；一个符号引擎把这些操作施加到 dependency map 上（认识论可信度模型 + 带 active/weakened/abandoned/resolved 标签的论辩框架 + 公开承诺集 + 依赖映射）。groundedness 检查与撤回查询都是对该结构的线性遍历，零 LLM 调用，且撤回满足 conflict-free 保证（撤回一个前提只移除恰好依赖它的结论）。

- 结果：在信念修正基准与 MemoryAgentBench 的 fact-consolidation 分片上超过预算匹配的 transcript-RAG 基线，把该分片单跳从 0.46–0.95 提升到 0.93–0.98；加了验证器后最弱的 7B 模型超过未加验证器的 GPT-4o；每查询成本与对话长度无关（prompt 稳定在约 0.8k token，而全上下文方案 66k–114k）；2000 轮时撤回查询 <1μs。
- 意义与局限：它把"轻量关系层"的形态具体化到可实现的程度，并免费提供精确撤回语义——正对本项目 S4 的可删除性缺口。局限：前提是每句话可归入八类之一；基准自带结构化更新元数据（作者用 GPT-4o 抽取做了缓解实验）；我只读到方法与形式化章节，命题证明与全部数字未核。我的评价：若本项目要把"一跳边"升级，升级成依赖/撤回关系比升级成图库更有证据支撑。

[A-04] Reliable Post-Retrieval Assembly for Agent Memory: Separating Evidence Extraction from Policy Execution（arXiv:2606.01435 v2，COLM 2026 Lifelong Agent Workshop poster，代码公开，评分 8.18）

- 做了什么：把"后检索装配"当作独立可靠性边界，量化"检索到了"与"用对了"的差距。
- 怎么做的：让 LLM 先把语义匹配的证据抽取为候选表示，再由一个独立阶段执行所需答案策略（如"取更新的那条"），从而把语义过滤、冲突消解、旧值抑制与最终生成解耦；在 MemoryAgentBench v3 的 FactConsolidation 分片与 LongMemEval 上做受控全流水线比较（相同 backbone、相同 top-10 证据、相同分块）。
- 结果：该分片明确声明"序号越大越新"，但已报告的最好结果仅 54% 单跳，全部 22 个已报告系统的多跳 ≤7%；解耦流水线在 262K 下达 82%/93% 单跳（gpt-4o-mini/gpt-4o）与 27%/41% 多跳。受控比较：单跳平均提升 10.8 个百分点，而仅替换最终策略执行器只贡献平均 2.0 个百分点（262K 处为 0）。LongMemEval 上无显著整体优势（26/45 vs 29/45，p=0.45），作者据此把结论限定在"带显式版本元数据的当前值问题"。
- 意义与局限：这是"该花力气治理检索与装配精度、而不是替换存储层"的量化支撑，并给出一个可引用的公共天花板（多跳 ≤7%）。局限：workshop poster 未经主会评审；作者自我限定范围；我未读到受控实验的统计细节。

2.3 写入、代价与风险：三个被低估的约束

[A-05] LSREP: A Longitudinal State-Replay Protocol for Evaluating Conversational Memory, with ICE v2 as an Audited Local-First Architecture（arXiv:2609.16730，2026-09-15，预印本，代码与冻结快照公开，评分 8.45）

- 做了什么：本报告中最重要的反向证据——一个刻意非向量的"本地优先"记忆架构作者，公开报告自己输给了纯向量 RAG。
- 怎么做的：提出纵向状态重放协议（有序重放 + 显式生命周期计划 + 重复探针 + 演进中的参考答案 + 机制保真检查）；以其自研 ICE v2（类型化存储、检索融合、动态上下文预算）为审计案例：私有单用户实例含 1,985 轮、219 个探针、52 个检查点上的 1,211 次观测；另有 LongMemEval 的公开端点对照诊断。
- 结果：在 LongMemEval 的匹配公开诊断中，ICE v2 明显输给纯 vector-RAG——evidence-only oracle 50.8% vs 72.8%，full-S 43.0% vs 69.5%，配对差分别为 -22.0（95% CI [-26.6, -17.4]）与 -26.5（[-31.3, -21.8]）个百分点，伴随保守拒答与严重的多会话/时序失败；在三个"普通密度"数据集上与 vector-RAG 平均质量差接近零且少选 32% 片段，但多耗约 6.6% prompt token。作者自己的机制保真审计承认：程序性检索本身有缺陷、若干机制未被触发、图结构的效用未被证明。
- 意义与局限：它示范了两件自建评测必须做的事（机制保真检查、公开端点对照），同时是对"零向量

库"最有力的反证。局限：单作者；主要实例不可外部复核；审计对象与被比较对象同一（因此可能低估"该架构类别"）；公开对照只用一个基准、一次配置。我的评价：这条必须写进决策依据的"反方陈述"里，它把"要不要上向量"的门槛从"哲学争论"变成"你在公开端点上掉多少分、以及你为什么认为你的工作负载不同"。

[A-06] Utility Under Attack: Agent Memory Poisoning and the Limits of Content Screening and Provenance Ranking（arXiv:2608.21230，2026-08-21，预印本，评分 8.33）

- 做了什么：把防御目标从"降低攻击成功率"改成"被投毒后记忆还剩多少价值"，并给出投毒比例—效用曲线。
- 怎么做的：威胁模型取最弱形态——攻击者只能通过非信任通道写内容（浏览网页、工具返回、共享命名空间），知道 agent 大致会被问什么并据此措辞但不做任何优化；系统自带四阶段写时内容筛查、以及"语义相似度 + 来源先验"加权的读时排名器。
- 结果：1.2% 的投毒语料把 LongMemEval 准确率从 0.850 打到 0.300（抹掉约三分之二效用）；那套写时筛查对间接提示注入召回 0.832、对含触发词的良性文本仅误报 1.5%，但对 360 条投毒记忆拒绝 0 条；随版本发布的来源权重与"无防御"统计上不可区分（p=0.80），提高权重后防御生效但机制变成排除而非偏好；当承载答案的证据本身来自非信任通道时，120 个问题中无一条证据记忆能通过排序，准确率跌到 0.0417。
- 意义与局限：这条攻击没有 payload、没有指令、没有触发词，因此"写时内容筛查"是结构性盲区而非检测器不够好——区分真假陈述通常需要文本之外的接地。对本项目的直接推论：安全不能寄托在写时过滤器上，而应落在来源、配额与人工确认。局限：单作者、被评测系统即作者自家系统（自曝负结果）；攻击非自适应，作者自称结果是下界；只在单一基准上做。

[A-07] Fortunate Recall: Ontology-Driven Memory Lifecycle Management for Persistent Coherence in LLMs（arXiv:2609.10413，2026-09-09，预印本（作者注明 under review），评分 8.18）

- 做了什么：用"确定性生命周期策略 + 仅两次 LLM 调用"的结构拿到高成绩，并通过预注册消融回答"到底哪一层在起作用"。
- 怎么做的：把抽取出的事实（关系边）分类到 10+1 个行为类别，类别决定衰减率、替代语义、到期逻辑与检索路由；生命周期层是确定性的闭式 log-score；随时间衰减的半衰期跨 4–19 天；slot-key 替代按归一化 (subject, attribute) 键触发，高置信矛盾把旧边置为非活跃、低置信保留双值软替代；事件时间有效性与前瞻激活分离；类别感知检索路由。
- 结果：在自建 LifecycleBench（516 题时序消歧）上 76.9%，高于 Mem0 61%、A-MEM 65.3%、MemoryOS 70.5%；完整 LongMemEval-S 500 题 75.2%。关键消融：把细分类本体换成三个"通用生命周期原语"+ 仅语义排序器，正确率统计上不变（-1.7pp，95% CI [-6.0, +2.7]），即承载正确率的是通用生命周期元数据；细分类本体承载的是校准（把下游幻造减半，12.0% vs 24.2%，p<0.001）。端到端上把对照的幻造率从 45.1% 降到 22.4%，同时在独立构建的 BEAM 上复现增益（46.8% vs 32.9%），增益集中在矛盾消解。
- 意义与局限：这是对本项目落地次序最直接的一条指导：先补通用生命周期元数据（时间戳、来源、活跃/失效标志、槽位键），再考虑细分事实类别。局限：基准与本体同出一手（自产自测风险，用预注册 + 独立基准迁移缓解）；LLM 抽取的粒度漂移未被量化（三判官一致性仅 κ=0.67）；代码与基准"发表后公开"，当前不可复核。

[A-08] Graph Memory for LLM Agents: At What Cost? A Comparative Evaluation of Query, Ingest, and Update Performance Across Graph Database Engines（arXiv:2609.23315，2026-09-20，预印本，评分 7.78）

- 做了什么：把"要不要上图库"从立场问题转成成本结构问题。
- 怎么做的：构造合成属性图（102 万节点，节点+边 534 万行）与 20 条查询负载，在三档规模上把一家面向本体的列式查询引擎与 7 个目的构建/图扩展型数据库（LoraDB、Ladybug、DuckPGQ、Memgraph、Neo4j、HugeGraph、FalkorDB）对比，报告查询延迟几何均值、批量写入吞吐、点更新延迟、答案正确率，并推导把 ingest/query 权衡表达为查询量函数的 TCO 模型。
- 结果：没有系统全面最快（原生图引擎在窄范围有界邻域上更快；列式在需要扫描或连接大部分图时更快）。主导成本差异不是查询延迟，而是"让数据变得可查询"的成本：批量 ingest 吞吐跨三个数量级（5.0k–4.3M 行/秒），交叉点显示在"每次数据刷新少于约 10^5 次查询"的工作负载下 ingest 成本主导总成本。
- 意义与局限：对"数十个记忆单元"的场景，这句话的含义是——上图的收益必须在查询侧有极大回报，才能覆盖它自己的写入与运维代价。局限：作者与被评测系统同源（利益相关，已披露）；102 万节点的规模与本项目差 4–5 个数量级，外推必须谨慎；未跑任何 agent-memory 任务。

[A-09] What Eviction Destroys: A Restore-Counterfactual Audit of Forgetting in Agent Memory（arXiv:2609.08279，2026-09-08，预印本，评分 8.05）

- 做了什么：提出"不可逆损失"作为预算—准确率之外的第三种评测口径。
- 怎么做的：对每个问题做配对干预——把该题 gold evidence 重新注入读时上下文并重跑同一 reader——再结合"该证据在淘汰后是否仍被保留"这一事实，把每个"oracle 可答却答错"的错误分成可恢复、不可逆、残余三类；在 LongMemEval-S 上比较四种淘汰策略在三个 token 预算、两种检索机制下的表现。
- 结果：80k 预算 + top-k 检索下，可被恢复修正的错误中不可逆占比 0.67–0.73（FIFO、随机、冗余感知），LLM 重要性淘汰为 0.60；8k 预算下四种策略均达 1.00；在强制注入 gold 的机制下按构造不存在可恢复错误，因此预算—准确率结果在未声明检索机制时不可直接比较；探索性的准确率匹配分析在 1.2–6 个百分点分辨率下未能检出差异（但能检出故意设置的破坏性对照）。
- 意义与局限：它同时给出两个对本项目有约束力的结论——写入与保留策略比检索算法更能决定不可逆损失，以及任何"回顾性重放"式自建评测都必须连同检索机制一起报告（这与项目文档自陈的"回顾性重放只是可达性证据"同向）。局限：单作者预印本；只在单一基准上；主导 reader 为单一模型；方法假设 gold evidence 可被识别并注入，这在开放域不总成立。

[A-10] Neuromem: A Granular Decomposition of the Streaming Lifecycle in External Memory for LLMs（arXiv:2602.13967，2026-02-15，预印本，评分 7.30）

- 做了什么：指出多数记忆评测假设静态（离线建库、固定状态查询），而真实场景是流式（新事实持续到达、插入与检索交错、状态在服务查询的同时演进），并给出对应的测试床与五维设计空间。
- 怎么做的：在"插入-检索交错"协议下评测，把外部记忆生命周期分解为五维：数据结构（分区式离散块 vs 层次式显式依赖）、写入归一化（粒度与保真度）、整合策略（冲突消解、容量衰减淘汰、结构精炼）、查询构造、上下文整合（重排或融合）；在同一条服务栈内替换各维度代表变体做受控消融，用 LoCoMo、LongMemEval、MemoryAgentBench 三个数据集报告 token 级 F1 与插入/检索延迟。
- 结果：作者总结为"复杂度的守恒"——系统通常在插入与检索之间转移计算债而非消除它；数据结构（是否显式建模依赖）决定可达的准确率天花板；性能通常随记忆在轮次间累积而退化，且时间相关查询始终是最难的一类；激进压缩与生成式整合主要是把成本在插入与检索之间搬移，准确率收益有限。
- 意义与局限：这是本报告对"规模"这一子问题唯一直接相关的学术证据，且对本项目有一句警示："记忆越多、精度越低"是被观察到的常态，与本项目实测的"案例 9→22 时噪声/查询 0.25→0.91"方向一

致。局限：我只读到设计与方法章节，"退化"的幅度与曲线形状未经表格核实；三个数据集均为对话/QA 类，与安全日志场景不同；未见第三方复现。

2.4 评测方法与基准：口径不可比是系统性问题

[A-11] Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions（MemoryAgentBench）（arXiv:2507.05257 v4，ICLR 2026 Poster，评分 8.60）

- 做了什么：把记忆评测从"单一问答准确率"改成四项能力口径——准确检索、test-time learning、长程理解、选择性遗忘——并把已有长上下文数据集改写为"记忆单元被逐步累积"的增量多轮格式。
- 怎么做的：以记忆科学与认知科学的经典理论界定四能力；把现有长上下文数据集与新构造数据集改造为增量格式；在同一基准上评测从朴素上下文、RAG 到外部记忆模块与工具型 agent 的多类系统。
- 结果：摘要层面的结论是"当前方法无一掌握全部四项能力"；可核的外部数字来自后续工作在 FactConsolidation 分片上的报告——最好结果仅 54% 单跳、22 个已报告系统的多跳全部 ≤7%（[A-04]），而符号验证器可把该分片单跳从 0.46–0.95 提到 0.93–0.98（[A-03]）。条件：该分片自带"序号越大越新"的显式更新元数据。
- 意义与局限：其中"选择性遗忘"与"test-time learning"正是 LoCoMo/LongMemEval 覆盖较弱的两项，也对应本项目"案例失效"与"库外内容应静默"。它是 ICLR 2026 接收、且被 2026 年多篇工作当作基线的公共刻度。局限：四项能力由论文自定而非社区共识；增量格式与真实日志流的分布差异未量化；我读到的是摘要与 OpenReview 页面，未见分项数字与基线配置。

[A-12] LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory（arXiv:2410.10813 v2，ICLR 2025，评分 8.38）

- 做了什么：给出 500 题的长期记忆基准，并把记忆设计拆成索引 / 检索 / 读取三阶段统一框架。
- 怎么做的：500 道人工精编问题嵌在可自由伸缩的用户-助手会话历史中，覆盖五项能力：信息抽取、跨会话推理、时序推理、知识更新、拒答；在三阶段框架下做受控实验，提出会话分解（值粒度）、事实增强的键扩展（索引）、时间感知的查询扩展（检索）。
- 结果：商业对话助手与长上下文 LLM 在"跨持续交互记忆信息"上出现约 30% 的准确率下降；上述优化同时提升召回与下游问答。条件：500 题、可扩展历史长度、LLM 判定式评分；基准与代码公开（参考实现见 [O-06]）。
- 意义与局限："知识更新"与"拒答"被写进公共评测口径的源头，正对本项目的冲突更新与"无匹配即【新案例】"；其"读取/装配"一环最容易被自建评测忽略，而 [A-04] 正是量化这一环的工作。局限：500 题规模小、单题波动影响大；依赖 LLM judge，摘要未交代基线细节；已知其 500 题池被多篇后续工作反复使用，存在同一池反复调参的风险。

[A-13] Evaluating Very Long-Term Conversational Memory of LLM Agents（LoCoMo）（arXiv:2402.17753，2024-02-27，评分 7.05）

- 做了什么：构造超长期对话数据集与配套评测基准，覆盖问答、事件摘要与多模态对话生成。
- 怎么做的：机器-人混合流水线生成对话，以 persona 与时序事件图接地，并让每个 agent 可分享/回应图像；人工标注者校验长程一致性与事件图接地。数据集每段平均 300 轮 / 9K token、最多跨 35 个会话。
- 结果：LLM 在理解长对话与把握长程时序、因果动态上存在困难；长上下文 LLM 与 RAG 能带来改进但仍显著落后于人类表现（摘要未给量化幅度）。
- 意义与局限：它是本领域最常被引用的基准，也是 Mem0/Zep 之争的战场。我核出一处口径不一致值

得记录：Mem0 论文称该数据集每会话约 600 轮、平均 26k token（[A-14]），而原论文自述"300 turns and 9K tokens on avg."，同一数据集两处描述相差近 3 倍。另一条观察：该基准的对抗（不可回答）类因缺 ground truth 被使用方普遍剔除（Mem0 与 Zep 双方都剔除，[A-14]、[I-07]），而同样地 LongMemEval 的 abstention 题在一个开源实现的公开记录里也被排除（500 题中只计分 470，[O-01]）——"无答案/不可回答"这一能力在公开报数中被系统性回避。局限：合成对话 + 人工编辑，与真实流量分布不同；仅 v1、无勘误版本。

[A-14] Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory（arXiv:2504.19413，2025-04-28，厂商自研预印本，评分 6.75）

- 做了什么：提出 Mem0 与其图记忆变体 Mem0g，并在 LOCOMO 上与六类基线系统（既有记忆增强系统、不同 chunk/k 的 RAG、full-context、开源记忆方案、专有模型系统、专用记忆管理平台）系统比较。
- 怎么做的：LLM 从持续对话中动态抽取、整合、检索显著信息；图变体在图上表示会话元素之间的关系。指标为 F1 / BLEU-1 / LLM-as-a-Judge（J，10 次运行取均值 ±1SD），评测单跳、时序、多跳、开放域四类问题。
- 结果（我逐节核读全文）：搜索延迟 p50 0.148s / p95 0.200s、总 p95 1.440s，而 full-context 总 p95 为 17.117s；full-context 的 J 最高（约 73%），论文原文承认它"can provide a slight accuracy edge"——记忆系统的收益在 token 与延迟（相对 full-context 的 p95 降约 91%）；Mem0g 以 J=68.44% 为全文最高，但摘要自报"比基础配置高约 2%"，且单跳为负增益、多跳零增益，图增益集中在时序与开放域；LangMem 搜索 p50 17.99s / p95 59.82s，论文自称"impractical for interactive applications"。
- 意义与局限：这是"加图只值约 2%"这一判断的原始出处——最常被引用为"图有效"的论文，同时是"图增益有限"的证据。局限（我核出的具体口径问题）：OpenAI 基线的延迟数字不含记忆抽取成本（原文明确写它处理的是"manually extracted memories"，且该成本"is not reflected in the reported metrics"），同时该基线在检索侧被"授予全部记忆"（intentionally granting privileged access）；对抗类被整类剔除；厂商自评、无同行评审、无第三方复现；随后被 Zep 公开质疑（[I-07]、[I-08]）。

3 工业线索结果

3.1 零向量库的产品先例

[I-01] Letta（MemGPT 系）. MemFS（git 支撑的记忆文件系统，默认无语义/向量索引）（官方一手文档，`Last-Modified` 2026-09-27；评分 8.93）

- 性质：官方一手文档（docs.letta.com/concepts/memfs）。
- 可核验内容：MemFS 是"the git-backed memory filesystem used by Letta agents"，记忆本身属于 agent 状态、存放在属于该 agent 的 git 仓库中，并投影为真实 checkout 供 agent 用普通文件工具读写；"All Letta agents use MemFS." 记忆按路径寻址（标签 system/persona 投影为 system/persona.md，选标签即决定它放在哪、是否留在上下文），每条记忆是带 YAML frontmatter 的 Markdown。加载规则：system/ 下的文件每轮进入系统提示；system/ 之外的文件不进上下文，但文件树本身常驻提示，"directory and file names act as signposts"，内容按需读取。检索（本课题核心）："MemFS does not include a semantic or vector index by default. Agents find memory in its Markdown files with normal file-search and read tools." 关键词搜索无需额外依赖，语义与混合模式需另装 MemFS Search mod 并先对记忆目录建索引；本地后端的会话转录检索目前只有全文匹配。版本化：每次记忆编辑都提交到该 git 仓库，提供版本历史与冲突解决。

- 宣称内容及其基线：文档为能力描述，无精度数字、有效果声明，故不涉及基线问题。
- 评价：这是本次调研中最强的单条支持证据——一个由 MemGPT 谱系公司交付、且"所有 Letta agent 都用"的生产系统，默认不做语义/向量索引，语义检索被降级为需显式安装的 mod。同时它给出一个常被忽略的成本优势：git 免费提供版本历史与冲突解决，而对照的托管服务需要专门实现类似的 revision 机制（[I-05] 所属产品线即有 memory revisions）。局限：文档未给记忆体量上限、加载延迟或精度数字，"文件树即 signpost"在数千条记忆时的可扩展性上限是该方法的关键未知量；且 system/ 每轮进系统提示，正是 [I-10] 记录的持久攻击通道，需要 [I-06] 的指令隔离约束来补偿。

[I-02] Anthropic. How Claude remembers your project（Claude Code 的 CLAUDE.md 与 auto memory ）（官方一手文档，页面未标日期；评分 8.73）

- 性质：官方一手文档（code.claude.com/docs/en/memory）。
- 可核验内容：记忆是纯文本文件、无嵌入，分两类并列——CLAUDE.md（用户写的持久指令）与 auto memory（Claude 依据用户的纠正与偏好自己写的笔记）。加载规则：CLAUDE.md 每会话加载；auto memory 每会话只加载"first 200 lines or 25KB"。文档还给出明确的性能—体量权衡："Size: target under 200 lines per CLAUDE.md file. Longer files consume more context and reduce adherence." 层级顺序为从文件系统根到工作目录逐级拼接而非互相覆盖（靠近启动位置的指令最后被读到），子目录文件按需加载，.claude/rules/ 可用 paths 字段做路径作用域加载。安全边界：项目级记忆文件中指向工作目录之外的 @path 导入被判为 external import，首次遇到会弹审批框，拒绝后保持禁用；用户级文件因"为自己所写"而被信任、不弹窗。治理：/context 可确认哪些记忆文件已加载；/doctor prompt-audit 扫描"为更早模型写的指令、指向不存在文件或命令的引用、互相矛盾的文件"并给建议，确认前不改动文件。
- 宣称内容及其基线：无精度宣称；200 行/25KB 属工程经验值。
- 评价：这是与对照基准几乎逐项同构的工业先例：路径即寻址、目录名充当 signpost、按需加载，对应本项目的确定性键与正则探针。更重要的是它给出可核验的体量经验法则并官方承认"超限会降低遵循度"——这是"记忆单元数量本身就是精度约束"的外部佐证，方向与本项目"必须治理检索精度"一致；/doctor prompt-audit 的存在则说明"记忆打架/过期"是真实且需工具化治理的问题，不是理论担忧。局限：页面无日期；200 行/25KB 是加载截断而非存储上限；该经验值不是基准测试结论，不能当作确定性检索的精度上限；文档未披露是否另有隐藏的向量索引。

3.2 图库的触发阈值与自建代价

[I-03] Zep. Why we built a graph database service for agent memory（Konig）（厂商工程博客，2026-08-27；评分 8.30）

- 性质：厂商工程博客（一手技术叙述，含失败复盘）。
- 可核验内容：Zep 原本运行在商用通用图数据库上，自述"After several challenging months of fighting outages and poor performance"，一名工程师用一到两个月原型化了基于内存邻接表的图引擎，2026 年上半年成为生产系统 Konig。核心论点是工作负载形状相反：通用图数据库假设一张大图（常驻内存、复杂遍历、schema 预先固定），而 agent 上下文是"millions of smaller graphs, one per user, team, or project, most in cold storage and all temporal and governed"。规模与延迟：从千图到数千万图，p95 检索保持 <100ms（本文档多处表述），端到端检索 <200ms，系统支撑每秒数千次变更与查询。成本模型：热图驻 RAM，周期性快照落本地 NVMe 与对象存储；"Cost tracks active graphs, not total graphs." 明确的能力让步："The design gives up global ordering and traversals that span every graph at once… a single retrieval targets one graph, and an agent reasons across many graphs

through separate calls." 检索实现：实体与边以紧凑整数索引数组 + 邻接表存储，关系遍历是指针解引用而非索引查找；多策略经 reciprocal rank fusion 融合，再经可选多样性重排或低延迟 LLM 重排，不需要另设 Elastic/OpenSearch 搜索集群。ANN 选型复盘：初始选择 DiskANN 是错的——因其索引条目内嵌邻居向量副本，导致库体积膨胀一个数量级。

- 宣称内容及其基线："通用图数据库在该工作负载下 handle poorly"——基线未披露（旧系统名称与旧数字均未给出）；"cost tracks activity, not capacity"属架构推论与规模外推，未见实际账单。
- 评价：这是本次调研中给"什么时候才真需要图库"最清晰的一条阈值：触发自建图数据库的条件是"数百万个图、每图一份记忆、绝大多数是冷的、且需按租户独立治理"，与本项目的"数十个记忆单元、单租户、单图"相差 4–6 个数量级。另有两处直接支持本项目形态：放弃跨图全局遍历是刻意的工程选择（一次检索只打一张图），以及 ANN 选型复盘给出了"零向量库"所规避掉的一类真实成本。局限：厂商自述性能、无第三方复现、未给硬件与图规模分布；索引与配置细节不可复现。

3.3 治理、删除与合规

[I-05] Anthropic. Use Claude's chat search and memory（记忆的保留、删除与企业管控）（官方一手帮助中心文档，页面只显示"Updated over a week ago"、未标绝对日期；评分 8.25）

- 性质：官方一手帮助中心文档（support.claude.com）。
- 可核验内容：记忆在 Free/Pro/Max 上默认开启，在 Team 与 Enterprise 上默认关闭、需 owner 开启。保留与删除规则逐条可核验：①"All memory will be retained in accordance with existing chat data retention policies."；②"When a conversation expires or is deleted, related memory entries generated from it won't be removed, but you can delete individual memories at any time."（源会话删除 ≠ 派生记忆删除，两者解耦）；③保留策略生效前生成的记忆不会被移除；④"Claude's memory is updated within 24 hours when conversations are created, modified, or deleted."（删除传播窗口 24 小时）；⑤Reset memory 会永久删除全部记忆（含 project 记忆），不可撤销。企业侧：owner 可为组织开启/关闭，关闭时"all existing memory entries for all users are deleted immediately"；"Owners can't view or edit a user's individual memories."；管理员可按组织保留策略导出全部记忆；记忆静态加密；Inkognito 会话不产生记忆但"remain available to owners through data export"。硬边界："Memory isn't available to organizations with HIPAA, public-sector, or custom data retention agreements."
- 宣称内容及其基线："记忆不会保存所有细节"等定性表述，无精度数字。
- 评价：三条最有决策价值的可核验事实：①托管记忆天然是"第二份副本"，法规意义上的被遗忘权需要跨两处操作，而对照基准里案例库即唯一副本、删一条就是删一条；②"能删"与"立即不影响推理"是两件事（24 小时窗口），只有组织级关闭才是立即且永久的强操作；③合规排除项是硬边界——在 HIPAA、公共部门或自定义保留协议下，"上记忆"与"满足合规框架"在当前产品形态下互斥，这对安全告警日志场景尤其重要。局限：页面无绝对日期（只能靠正文提到的 2026-09-09 旁证）；企业版与 legacy 两套机制描述交织，本次未逐段区分；未给记忆条数上限、检索通道或任何精度数字。

3.4 精度数字的披露与不可比

[I-04] Zep. Research：Benchmarks for agent memory（完整评测口径披露）（官方一手技术报告页，页面未标日期；评分 7.35）

- 性质：官方一手技术报告（基准结果页）。所有数字为厂商自我报告，无第三方复现。
- 可核验内容：LoCoMo 94.7%（1,459/1,540），检索延迟 p50 87ms / p95 155ms，上下文中位 5,760 tokens；分题型 single-hop 96.4%、temporal 95.6%、multi-hop 94.0%、open-domain 79.2%。

LongMemEval 90.2%（451/500），p50 104ms / p95 162ms，上下文中位 4,408 tokens；分题型 multi-session 83.5%、knowledge update 93.6%、temporal reasoning 90.2%、single-session assistant 96.4%。口径披露完整：reader 与 judge 均为 gpt-5.4（judge 用 chain-of-thought grading）；检索深度 20 edges / 10 nodes / 10 episodes / 5 thread summaries / 5 observations + cross-encoder reranking；两个基准均"0 failed tests"。主结果用多作用域并行检索并在客户端组合；另给 auto search 单调用方案：LoCoMo 86.5%、p95 173ms、上下文中位 2,680 tokens（比多路方案小 53%）。架构归因：热图以邻接表与 CSR 矩阵驻内存，向量与 BM25 索引并列。

- 宣称内容及其基线：如"production agents need all three"属定性；精度数字的基线（读/判模型、检索预算）已披露，这一点优于多数同行。
- 评价：这是三方中"精度数字必须看分母"的正面示范，也提供了本报告最可比的一组"精度—延迟—token"三元数字。两个对本项目直接有用的对照：①图原生厂商的检索也是混合的（向量 + BM25 + 图算法），并非纯图；②"较少上下文 + 较简单检索"的实测代价是准确率 94.7% → 86.5%（-8.2 分），而 context 减半——这是"轻量检索够不够用"目前最有量化锚点的一条外部证据。局限：厂商自测、无第三方复现；未给 judge 不一致的置信区间；页面无日期，无法判断这组数字何时测得、是否已被更新模型栈取代（这是本条目最大的时效性缺口）；multi-scope 依赖客户侧组合；auto search 与主结果不是同一配置，不可混用。

[I-07] Zep. Lies, Damn Lies, & Statistics: Is Mem0 Really SOTA in Agent Memory?（厂商工程博客，页面未标发布日、正文含 2025-05-12 的更正说明；评分 7.18）

- 性质：厂商工程博客——竞争对手方的技术质疑，含指向自家复算代码的链接。
- 可核验内容：声称以"correct Zep implementation"重跑后 Zep 的 J = 75.14% ± 0.17（并称 10 次运行），对比 Mem0 论文为 Zep 报告的 65.99%，约 10% 相对提升；页面保留一条更正说明早期版本算错了 Zep 的分数。延迟：Zep 搜索 p95 0.632s（并发搜索，测量条件写明为 AWS us-west-2 经 NAT），对比对手为其报告的 0.778s（Zep 称因对手用串行搜索而被抬高）与 Mem0g 的 0.657s；承认 Mem0 基础版 0.200s 更低，但指出那不是同类比较（基础版用更简单的向量存储/缓存、无关系能力，且准确率在 Mem0 各变体中最低）。对基准的批评：会话平均约 16,000–26,000 tokens，落在现代 LLM 上下文窗口内，并直接引用对手自己的数据指出 full-context 基线 J≈73% > Mem0 最佳≈68%；Category 5 因缺 ground truth 不可用、双方都剔除。对对手复现的指控：用户模型错误（把 user 角色同时给了两个参与者）与串行而非并行搜索。
- 宣称内容及其基线："Zep outperforms Mem0 by 10%"、"LoCoMo is a flawed evaluation"——争讼一方的口径，基线是自建重跑，代码链接指向自家仓库。
- 评价：这是"基准数字不可跨厂商比较"证据链的一半，也证明厂商的评测结论会被对手以代码为依据反驳。局限：利益相关方；该文自身也曾算错并更正，说明此类重跑同样脆弱；延迟测量含跨区 NAT，与对手口径不统一。

[I-08] getzep/zep-papers issue #5：Revisiting Zep's 84% LoCoMo Claim: Corrected Evaluation & 58.44% Accuracy（GitHub issue，创建 2025-05-08、关闭 2025-05-19；评分 7.05）

- 性质：对手方在 Zep 仓库公开提出的质疑（一手正文，含复算脚本链接）；经 GitHub REST API 读取标题、状态、时间与完整正文，3 条评论未逐条读。
- 可核验内容：主张 Zep 的 84% 分数把本应排除的对抗类问题也算进去了，因此"人为抬高约 25.56 个百分点"；在一致条件下（仅取前四个有效类别）复算得 Zep = 58.44% ± 0.20，而 Mem0 论文为旧版 Zep 算法报告的是 65.99% ± 0.16。指控基准配置不一致（称对方修改了 judge prompt、变更了模板变量，并给出具体代码行链接）。对两项质疑逐一回应，并称 Mem0 当年使用的 zep-cloud==2.10.1 不

支持 created_at 参数，该参数在对手论文发表后才于 2025-05-01 的 zep-cloud==2.12.1 加入，因此"事后 SDK 更新不能回溯性地证明当时的评测有缺陷"。

- 宣称内容及其基线：结论由竞争对手方按自家口径复算得出。
- 评价：与 [I-07] 构成同一争议的对立两侧——同一基准、同一对手系统，公开分数依次为 84% → 75.14% → 58.44%，每一步都伴随方法学指控。这是本报告能给出的最完整的一条"记忆系统公开数字不可比"证据链，同时它示范了一个可贵的工程习惯：质疑必须附带可复算的代码与输出。局限：利益相关方；issue 已关闭；我未读到 3 条评论全文，也未独立复现任何一方的脚本。

3.5 记忆攻击面与检索计价

[I-10] OWASP Gen AI Security Project. Memory Is a Feature. It Is Also an Attack Surface.（MemoryTrap，含 ASI06 与 LLM08:2025）（标准组织博客，2026-05-13；评分 8.30）

- 性质：标准组织博客，含一条具体漏洞披露的一手叙述（作者为 ASI06 条目负责人、Cisco 资深安全研究员）。
- 可核验内容：OWASP Agentic Top 10（2025-12-09 发布）单列 ASI06: Memory & Context Poisoning。2026-05-13 的博客披露 Cisco 研究的 MemoryTrap：只需 clone 一个仓库并批准安装依赖，即可污染持久记忆、hooks 与系统提示；并记录 Claude Code v2.1.50 把用户记忆移出系统提示作为修复。同一项目的 LLM08:2025 把"向量与嵌入弱点"单列（未授权访问、嵌入反演、投毒），缓解要求向量库内严格分区。
- 宣称内容及其基线：属缺陷披露叙述，而非受控实验；未给影响面估计与完整复现条件。
- 评价：它把"记忆进系统提示"定性为已被走通的持久攻击路径，而修复方式恰恰就是移出系统提示——这与 [I-06] 厂商侧独立给出的"记忆绝不进系统提示"完全一致，构成对照基准的一条硬约束：任何高信任通道一旦承载可写记忆，就把攻击面做成了持久化的。局限：博客非同行评审；OWASP 条目页本身未标日期，我读到的是发布公告与博客正文。

[I-06] Zep. Defending Agent Memory Against Poisoning（厂商工程博客，2026-09-02；评分 8.15）

- 性质：厂商工程博客（安全工程一手叙述）。
- 可核验内容：定义"Memory poisoning is prompt injection with persistence"；边界主张"模型不是安全边界……你能强制执行的边界是控制每次写入、每次读取与每个外部动作的应用代码"。引用 MINJA、AgentPoison、PoisonedRAG 三项攻击；枚举攻击目标（存假事实、用伪造的更新版本替换正确事实、存入会被后续模型当指令执行的文本、用同一账号重复同一主张制造伪佐证、灌库把真实内容挤出检索）。指出时间戳是最常用杠杆——许多系统按 recency 消解冲突，攻击者提交带假事件时间的声明即可让存储替换已验证事实；对策是把"来源声明的事件时间"与"应用接受记录的时间"分开存。提出五阶段传播模型（写入 → 存储 → 检索进提示 → 被模型采纳进推理 → 作为外部动作执行），并指出"不把上下文泄露给另一用户"这类伤害不需要任何工具调用。控制清单：受控写入（类型化操作 vs 通用 remember(text)——前者限制单次输入能改变什么，后者可存任意文本）、按来源与时间窗限速、外部溯源与血缘、作用域检索（先按用户/工作区/来源/审核态过滤，再做语义排序）、指令隔离。
- 宣称内容及其基线："None of the controls below removes the risk on its own"属自我限定；"Zep 能按来源过滤"是产品能力陈述，未给实现细节与对召回的影响。
- 评价：这是工业侧对对照基准最有操作性的一份"设计规格"，其中三条恰好是符号化结构化记忆天然具备的形态——类型化写入、先过滤后排序、记忆不进系统提示。特别是"先过滤后排序"给了确定性筛选一个独立于精度的安全理由：过滤发生在排序之后时，被排除内容仍会占据候选位，灌库攻击因此有效。局限：工程博客非同行评审；未提供任何防御有效性度量（无攻击成功率对照、无检测误报

率）；三项攻击只给名字与一句话概括，未给论文链接或实验条件（本次未去核验原始工作）；厂商是所讨论系统的提供方，存在利益倾向。

[I-09] Amazon Web Services. Amazon Bedrock AgentCore Pricing（Memory 段）（官方一手定价页，页面未标日期；评分 7.68）

- 性质：官方一手定价页（与 devguide 的 Memory 文档配套）。
- 可核验内容：Memory 段列出短期记忆 $0.25 / 千事件、长期存储 $0.75 / 千条·月、检索 $0.50 / 千次——写入与检索分别计费。配套 devguide 说明记忆分短期与长期，长期抽取策略可选内置或自管；并声明"可为提升本客户服务体验而使用与存储你的内容"。
- 宣称内容及其基线：数据使用声明为定性表述，未给保留期限。
- 评价：记忆服务第一次把"写入事件数""存储条数""检索次数"拆成三个独立计费维度，其中检索是独立账单项。这与 [I-04] 所属厂商的做法（检索/存储/图存储计 0 credit、只按写入字节计费）方向相反，说明"检索频率是不是成本中心"取决于选型：自建/符号检索下检索是 CPU 成本、可无限重试；托管平台下检索是账单项——这是"治理检索精度"在不同形态下预算含义不同的直接证据。局限：页面未标日期；AgentCore 的配额页在本次多次尝试中均未取到（重定向或 JS 壳），因此条数上限、保留期与速率限制本报告无法核验。

4 开源线索结果

> 本轨道的取证通道受限：本机无法 `git clone`（`github.com` 与 `raw.githubusercontent.com` 均不可达），全部仓库事实经 `api.github.com` 的元数据、递归文件树与文件原文端点取得。因此**没有一条经过本地构建或运行**；下文凡称"读到代码"均指读到该文件的 API 返回原文。许可信息除注明者外来自 API 的 `license.spdx_id` 字段，未逐字核对 LICENSE 正文。

[O-01] akitaonrails/ai-memory（Rust 记忆服务；MIT；默认分支 `main`，`pushed_at` 2026-09-27；评分 8.85）

- 检查版本与方式：API 路线（元数据 + 完整文件树 877 路径 + 读取 Cargo.toml、docs/vector-backend-policy.md、docs/benchmarks/longmemeval-s-2026-09-01-fts.md 原文）。未取 commit 短哈希。
- 实现了什么：面向编码 agent CLI 的长期记忆服务。存储为 SQLite + git 管理的 markdown，markdown 是唯一真相源；查询路径 memory_query 组合 FTS5 + 词法实体匹配 + 图邻居扩展 + 可选向量 RRF + 原始观察回退，每条通道经同一有界候选窗贡献，融合之后再做一次"页面权威度"调整，使"语义相似度不能单独把某个会话页判定为比一条已维护的决策更权威"。向量以打包向量存于 SQLite、查询时做暴力余弦。
- 本次亲验的关键一手材料（我独立读取并核对原文）：docs/vector-backend-policy.md 明确写"sqlite-vec is deferred intentionally, not rejected"，理由是"expected corpus is usually hundreds to low-thousands of latest wiki pages per project"，并给出可执行的升级触发条件：①每项目最新嵌入页数经常超过约 5k–10k；②memory_query 在 FTS 与图路径已优化后，p95 仍因向量评分超过约 150–250ms；③性能分析显示查询时点积消耗可观 CPU（尤其并发 MCP 调用下）；④真实检索评测显示向量结果相对 FTS5 + 实体 + 图有实质提升，例如 +5–10% recall@5；⑤迁移可在不重置用户数据的前提下从既有嵌入表回填；⑥派生索引可被丢弃并从 markdown + 嵌入重建，修复过程绝不删除源 wiki 文件。并明确列出非触发条件："Do not add sqlite-vec just because vector databases are conventional, or because it feels architecturally cleaner." 另有一份零 LLM 的公开基准记录（docs/

benchmarks/longmemeval-s-2026-09-01-fts.md，commit 0ac0dcf8…，mode: zero-llm，470 题计分、30 道 abstention 题被排除）：overall hit@1 0.534 / hit@3 0.647 / hit@5 0.668 / hit@10 0.696；分片 knowledge-update hit@1 0.583、temporal-reasoning 0.535、multi-session 0.446、single-session-assistant 0.679、single-session-preference 0.300；注释说明 hit@k 是"任何证据会话进入前 k"（即多数系统公布的 Recall@k），recall@k 是找到的证据会话比例，且无法归属的页面永不计分。

- 代码质量与活跃度：pushed_at 为检查前一日；Cargo.toml 版本 2.4.1、有 test-support 共享测试 crate 与独立 A/B 评测 harness；测试目录成体系（含 fts_drift_status.rs、embed_backfill.rs 等）。stars 8,496（弱信号）。
- 评价：这是本次开源轨道中对本课题最有决策价值的一条：它是唯一把"现阶段不上向量库"写成可执行判据（触发阈值 + 非触发条件）的项目，并且给出了零 LLM 的纯符号检索在公开基准上的实际水平——hit@1 0.534 / hit@5 0.668。与对照基准相比有两点直接可比：其触发下限（5k–10k 页）比本项目的 22 个案例高 2–3 个数量级；其"融合后再做权威度调整"的设计，与本项目"加权打分 + 人工确认"在意图上一致（都不让相似度单独决定什么算权威）。局限：单一项目自评、无第三方复现；基准记录是其自建 harness 产出，不可直接与厂商数字比较；其 hit@k 口径是检索层而非端到端问答。

[O-02] mem0ai/mem0（记忆层厂商；Apache-2.0；`pushed_at` 2026-09-25；评分 8.55）

- 检查版本与方式：API 路线（元数据 + 递归文件树 2210 路径 + 读取 memo0/utils/scoring.py、mem0/memory/notices.py、mem0/memory/oss_notices_config.json、mem0/memory/main.py 等原文）。未取 commit 短哈希。
- 实现了什么：库 + 自托管服务 + 托管平台三层。向量库是构造期必需（Memory.__init__ 无条件构建 VectorStore 工厂）。融合公式经我读取 mem0/utils/scoring.py 原文确认：combined = (semantic + bm25 + entity_boost) / max_possible，其中 ENTITY_BOOST_WEIGHT = 0.5，BM25 经查询长度自适应的 sigmoid 归一化到 [0,1]；并且文档字符串明确写"Threshold gates the semantic score BEFORE combining — candidates below the threshold are excluded even if BM25/entity would boost them." 即阈值只门控语义分。mem0/memory/notices.py 中我读到三个规模/性能阈值常量：SCALE_MEMORY_COUNT_THRESHOLD = 2000、SCALE_TOP_K_THRESHOLD = 50、PERFORMANCE_SLOW_QUERY_THRESHOLD_SECONDS = 2.0——即厂商自己认定的三个"该考虑换方案"的点。附带说明：随包发布的 oss_notices_config.json 里所有提示项 enabled: false 且文案为空，真实文案经远端配置拉取（TTL 3600s、variant_split: 0.5），即这类提示是远端灰度控制的功能。
- 代码质量与活跃度：pushed_at 为检查前 3 天；有配套论文与单独开源的评测 harness（[O-07 类]）；开放 issue 数 752（偏高但与体量相称）；stars 66,117。
- 评价：读代码得到的两个事实对选型有直接影响：①这家厂商的默认检索不是纯向量而是三信号加性融合，且 BM25 被显式归一化后参与打分——"需不需要混合检索"在头部厂商这里是既成事实；②厂商自己把 2,000 条记忆 / top_k 50 / 2 秒查询设为"规模告警"阈值，可作为"何时该重新评估检索架构"的外部参照点。局限：默认向量 store 的具体 provider 本次未确认；许可与部分配置未逐字核对；OSS SDK 与平台能力不等价（该厂商自己在文档与本仓库中标注了平台私有优化），故 README 上的基准数字不可用于评估开源版。

[O-03] letta-ai/letta → letta-ai/letta-code（原 MemGPT；Apache-2.0；评分 8.30）

- 检查版本与方式：API 路线（两仓库元数据 + letta-code 递归文件树 2553 路径 + 读取转录检索打分实现原文）。两仓 HEAD commit 均已取（letta：5bcdd177…，2026-09-10；letta-code：pushed_at 2026-09-28）。

- 实现了什么：一次重要的仓库迁移——letta-ai/letta 的 main 分支已只剩 15 个文档路径（源码迁至 letta-ai/letta-code），而旧仓 stars 24,921、新仓 3,444，按 star 数选型会直接选到已退役的仓。新仓 2,553 条路径中匹配 embed|vector|graph 的路径数为 0；agent 记忆是 git 版本化的文件系统（对应其官方文档 [I-01]）；本地会话转录检索的打分函数为位置加权的合取式关键词匹配（读到的实现形如 score += index + max(0, 50 - term.length)），即位置越靠前、词越短得分越高——一种纯词法的确定性排序。
- 代码质量与活跃度：letta-code 在检查当日仍在推送；旧仓 CI 只跑 issue 分流。
- 评价：两条对选型有约束力的发现：①仓库热度和实际维护状态可能背离——这条对任何"看 star 数决定用哪个记忆框架"的流程都是直接警告；②新仓代码里没有任何 embed/vector/graph 路径，与该厂商官方文档"默认不含语义/向量索引"互相印证，使"零向量库作为生产默认形态"从一个文档声明升级为文档 + 代码双证据。局限：仅路径级与单函数级证据，未做完整代码审阅；许可来自 API 字段而非 LICENSE 正文；本地转录检索只有全文匹配是文档陈述与代码一致，但未测其效果。

[O-04] getzep/graphiti（时序知识图谱记忆框架；Apache-2.0；HEAD commit `6b4b56f…`，2026-09-27；评分 8.15）

- 检查版本与方式：API 路线（元数据 + 文件树 + 读取 graphiti_core/search/search.py、search_config.py、edge_operations.py 原文）。
- 实现了什么：Zep 的开源内核。对第三方图库是硬依赖：README 的 Requirements 与 .env.example 指向 Neo4j 5.26 / FalkorDB / 或 Neptune + OpenSearch 组合，即要跑起来必须先有一个独立的图数据库。检索默认为混合，默认排序器为 rrf（reciprocal rank fusion），另有 node_distance、mmr、cross_encoder 等可选项；源码文件头带 Apache 2.0 声明。
- 代码质量与活跃度：最后提交为检查前一日；CI 存在（lint / unit_tests / typecheck，另有 codeql、dependabot、stale 与 issue 自动分流）；开放 issue 524；有配套论文（arXiv:2501.13956）。
- 评价：这条给出"图记忆"路线的实际部署代价的最低值——不是"加一个字段"，而是"必须运维一个图数据库"（或改用托管服务）。它与 [I-03] 该厂商自述"通用图数据库在我们的工作负载下 handle poorly、于是自建了 Konig"合起来读，说明即便在图派厂商内部，图库也是一个需要持续投入的基础设施决策，而不是检索层的可选项。局限：未运行，仅代码与依赖级证据；未核实际检索效果；未统计部署复杂度（容器、迁移、备份等）。

[O-05] snap-research/locomo（基准参考实现）（评分 7.53）

- 检查版本与方式：API 路线（文件树 + 读取 task_eval/evaluation.py 全文 293 行）。许可未声明（仓库树含 LICENSE.txt 但本次未读其内容）；未取 commit。
- 实现了什么：LoCoMo 的官方评测脚本。我逐行核对后的关键事实：该脚本只导入 regex、json、string、unicodedata、typing、numpy、collections、os、bert_score、nltk.stem.PorterStemmer——没有任何 LLM API 客户端（无 openai / anthropic / requests / api_key），核心判分是 Porter 词干化后的 token 级 F1 与集合相等精确匹配，另引入神经指标 BERTScore 作为补充。另有 scripts/ 下 7 个脚本按被测模型（gpt / claude / gemini / hf_llm / rag_gpts 等）分别封装。
- 代码质量与活跃度：仓库含 Python 3.9/3.10 的 __pycache__ 残留，说明长期以该环境运行；未见 CI。
- 评价：这条的意义不在基准本身，而在于它证明"主流记忆基准可以完全用确定性指标判分"——这为本项目采用确定性评测路线提供了直接的外部先例，也说明后来厂商普遍改用 LLM-as-judge 报数是一次口径变更，而非唯一可行做法。需要注意一处精确性：脚本确实引入了 BERTScore（一种神经相似度指标，本地模型），所以"零 LLM 调用"应表述为无 LLM API 判官，而不是"完全无数值模型参与"。局限：许可未确认；未运行脚本。

[O-06] xiaowu0162/LongMemEval（基准参考实现）（评分 7.70）

- 检查版本与方式：API 路线（读取 README.md 与 src/evaluation/evaluate_qa.py 全文 134 行）。许可未声明（未取元数据端点）；未取 commit。
- 实现了什么：LongMemEval 的官方代码。判分走 LLM-as-judge：max_tokens=10，判定方式为 'yes' in response.lower()——即由一名 LLM 返回极短回答并做字符串包含判断。检索侧把 BM25 与稠密检索并列为 flat-* 基线，即符号检索在该基准里是一等公民而非陪跑。README 给出两档环境（requirements-lite.txt 只跑评测、requirements-full.txt 含记忆系统实验），并有数据清理公告。
- 代码质量与活跃度：有 ICLR 2025 接收；数据清理公告说明维护方在乎标注质量。
- 评价：读代码比读摘要多得到三件事：①判分规则相当宽松（temporal 题豁免 off-by-one、knowledge-update 允许答案同时含旧信息、preference 题不必覆盖 rubric 全部要点），因此任何引用该基准分数的说法都应连同判分口径一起引用；②BM25 与稠密检索被并列为基线，从基准设计层面否定了"必须向量化"的默认假设；③该基准的评测脚本被后续基准逐字复制（[O-08]），所以拿多个基准互相印证时要注意它们不是独立证据。局限：许可未确认；判官的具体模型与提示词未在本次读取范围内完全确认。

5 交叉对比

议题	学术线索	工业线索	开源线索	我的判断与依据
向量层是否必要	必要性随工作负载变化：同一底座在 LoCoMo 上 BM25 单通道最强、在 LongMemEval 上稠密通道有增量（[A-01]）；但非向量架构在公开端点落后 22–26.5 个百分点（[A-05]）	图原生厂商的检索同时持有向量与 BM25（[I-04]）；头部记忆厂商默认三信号加性融合（[O-02]）	生产级框架默认无向量索引（[O-03]/[I-01]）；一个项目把它写成可执行触发条件（5k–10k 页 / p95 150–250ms / +5–10% recall@5）（[O-01]）	向量层不是必需品而是"按需触发件"，但触发条件必须由自己的查询类型与规模测量决定。四类证据中唯一明确反对的是 [A-05]，其场景（LongMemEval 级历史、公开端点）与对照基准相差数个数量级，不足以推翻"现阶段不上"，但足以要求把重访条件写成可测阈值。
图/关系层值多少	图库决策变量是 ingest 成本而非查询延迟（[A-08]）；只保留三类关系 + 一跳扩展即可同 backbone 最优（[A-02]）；依赖表形态可做到零 LLM 线性遍历（[A-03]）	头部厂商自报"加图约 2%"且只用共现边做排序提升（[A-14]；另见附录 B 的 Mem0 Graph Memory 文档旁证）；自建图库的触发门槛是"数百万小图、多数是冷的"（[I-03]）	图记忆框架对第三方图库是硬依赖（[O-04]）	共识：关系层值得做，图库不值得在这个规模上做。三条线互相印证，且都指向"稀疏、判别性、带确认的关系"优于"稠密图结构"。
精度治理该做什么	收益主要在把"证据识别"与"策略执行"解耦（[A-04]）；写入侧通用生命周期元数据承载正确率、细分类本体买校准（[A-07]）	先过滤后排序（安全理由）、类型化写入、记忆不进系统提示（[I-06]、[I-10]）	融合后再做权威度调整，拒绝让相似度单独定权威（[O-01]）	共识且可操作：治理对象是"装配 + 写入元数据 + 过滤次序"，而不是"换一个更贵的索引"。
无答案/拒答能力	已被写进公共口径（[A-12] 的 abstention、[A-11] 的选择性遗忘），但公开报数普遍剔除该类（[A-14]、[I-07]、[O-01] 只计 470/500）	厂商文档未把拒答作为能力项披露	判分规则宽松，且对抗类不可用（[O-05]、[O-06]）	证据不足：所有人都在口径里承认它重要，但公开数字里几乎看不到它——这正是本项目前瞻测试 3/3 失败的那一项，且没有外部基线可对标。

议题	学术线索	工业线索	开源线索	我的判断与依据
删除与合规	本轨道未找到"记忆存储删除权"的学术材料（检索只命中模型权重卸载类 unlearning）	托管记忆删除与源数据解耦、传播有窗口、部分合规框架下功能整体不可用（[I-05]）	git 版本化天然提供历史与冲突解决（[O-03]）	工业侧独有证据。对本项目意味着：自建单副本形态在删除语义上天然更简单，这是"零托管"路线一个未被计入的合规优势。
公开基准数字能否用于选型	基准自身存在答案键与判分效度质疑（[A-13] 的口径不一致、[A-14] 的基线口径问题）	同一对抗双方给出 84% → 75.14% → 58.44% 三套数字（[I-07]、[I-08]）；两厂商口径逐项不同、1.5–2.2 分差落在口径差异内（[I-04]）	判分脚本一为确定性、一为宽松 LLM 判官，且被后续基准逐字复制（[O-05]、[O-06]、[O-08]）	分歧：厂商侧的数字不能用于跨厂商选型；我采信"口径披露完整的一方更可作为方法论参考"（[I-04]），但不采信其分数能代表我的场景。

- 共识（三条线互相印证）：①关联/关系层值得保留，但形态应是稀疏、判别性、可确认的，而不是图库；②检索精度治理的重点在装配与写入元数据，而非索引替换；③记忆单元数量本身构成精度约束（[I-02] 的 200 行/25KB、[O-02] 的 2000 条、[O-01] 的 5k–10k 页），因此"规模触发重估"应当被写成可测阈值而不是感受。
- 分歧：①向量层必要性——[A-05] 给出负面证据，[A-01]/[O-01]/[I-01] 给出"按需触发"的正面处理。我判定后者证据更强，理由不是数量而是可比性：[A-05] 的对照是在公开端点、以端到端准确率进行的，而 [A-01] 与 [O-01] 给的是检索层指标与可测阈值；两者可以同时成立——即"在 LongMemEval 级负载上纯符号检索会掉分，但在本项目规模上它不是瓶颈"。要分清这两句话，需要的是本项目自己按查询类型分层的测量，而这恰好尚缺。②公开分数——厂商之间的互诉说明该赛道的数字生态不可信；我倾向采信披露更完整（给出分母、模型、检索预算、延迟分位）的一方作为方法论样板（[I-04]），但拒绝任何厂商的分数作为对本项目场景的预测。
- 证据不足：①"确定性规则抽取 vs LLM 抽取"没有任何论文做直接对比实验，粒度漂移也无人量化（学术轨道专门为此标注了材料稀薄）；②"记忆单元数量增长 → 检索精度退化"没有一篇以此为主动实验的工作（[A-10] 是最接近的，但它只给出定性判断且我未读到结果章节）；③真实用户侧的退化报告完全缺失——开源轨道一条 issue/PR 都没读到，所以"规模上去以后精度会不会坏"目前只有设计约束与合成实验两类证据；④被遗忘权/合规删除在学术轨道没有材料。

6 综合评价与观点

> 本节是我的判断，与前述来源陈述明确区分。凡是来源主张我都标了键；凡是我推断的都给出理由。

6.1 证据支持的结论

1. "现阶段不必上向量库/图库"这个结论成立，但它成立的机制与项目文档目前的表述不同。项目文档倾向于用"入口召回 99–100%""扩展新发现 110 条"这类自评数字支撑；而外部证据支持的是一个更弱但更稳的命题：在这个规模与工作负载下，向量层与图库的增益无法被任何一方证明，而它们的代价是可测的——图库的代价是 ingest 与运维（[A-08]、[O-04]、[I-03]），向量层的代价是索引构建与存储膨胀（[I-03] 的 DiskANN 复盘）、以及检索被计价（[I-09]）。同时，"零向量库作为产品默认形态"有文档与代码双重先例（[I-01]、[O-03]）。
2. 关系层值得保留，但形态应当更保守。 三份独立工作都指向同一形态：关系只要稀疏、判别性、可确认的少数几类（[A-02] 只有 merge / supersession / contradiction 三类且只做一跳；[A-03] 干脆是一张带 standing 标签的依赖表；[I-03] 的工业版本放弃跨图全局遍历）。这与项目第二轮"只有稀疏跨簇

边通过判据、稠密簇内边噪声放大"的实测完全同向。

3. 真正被外部证据支持的"下一步投入"是写入侧，而不是检索侧。 三条：①通用生命周期元数据（时间、来源、活跃/失效、槽位键）承载了正确率收益，细分本体只买校准（[A-07]）；②不可逆损失主要由保留/淘汰策略决定，且大部分不可由更好的检索挽回（[A-09]）；③"证据识别"与"策略执行"解耦后的增益远大于替换最后一步的算子（[A-04]）。这三条共同指向"把字段与生命周期补对，比把打分器调好更值钱"。
4. 精度治理必须按"检索 + 装配"两段来做，并且过滤要发生在排序之前。 前者有量化支撑（[A-04]）；后者既是安全要求（[I-06]：过滤发生在排序之后时，被排除内容仍占据候选位，灌库攻击因此有效），也是 [O-01] 的实现选择（融合后再做权威度调整）。项目现有的"确定性加权打分 + 档位门控"在结构上已经符合这一形状，问题只在参数（通用词泄漏），不在架构。

6.2 被高估或过度声称的部分

- 厂商的基准分数不可用于选型，且这一条有完整证据链。 同一基准、同一对手系统，公开数字依次出现 84% → 75.14% → 58.44%，每一版都伴随方法学指控（[I-07]、[I-08]）；两厂商的口径（读模型、判模型、检索预算、context 大小）逐项不同，1.5–2.2 分的差距落在口径差异量级内（[I-04]）。任何以"某产品 LoCoMo 92.5 / LongMemEval 94.4"为依据的选型判断都是不可靠的。
- Mem0 论文的"26% 相对提升 / 91% 延迟降低"是最常被引用、也最容易被误读的一组数字。 我核读全文后的具体问题：它的 OpenAI 基线在延迟里不含记忆抽取成本（原文明确写该成本"is not reflected in the reported metrics"），而抽取恰恰是记忆系统的主要成本项之一；同时该基线在检索侧被授予全部记忆。这两处偏差方向相反，使"相对提升"既不能作为端到端对比，也不能作为单侧劣势的修正。
- "图记忆有价值"这一印象主要来自论文标题与厂商材料，而不是它们自己的数字。 那篇论文自报"加图约 2%"、单跳为负增益、多跳零增益（[A-14]）；另一家厂商的图只做共现边与排序提升。把"图"当作收益来源是有文献依据的；把"图库"当作收益来源没有。
- 厂商的精度数字在时间维度上不稳定。 同一厂商同一基准，2025 年的页面给的是"相对全上下文基线的增量"（并诚实披露某一类别下降 17.7%），当前页面给的是绝对准确率 90.2%，读/判模型从 gpt-4o 系换到 gpt-5.4——同一产品的"能力数字"在一年内被换了口径与模型栈（[I-04] 及其页面的时效缺口）。
- "记忆越多越准"没有证据，反向证据反倒有一处。 公开实现给出的体量经验上限（[I-02] 的 200 行/25KB）、厂商自己的规模告警阈值（[O-02] 的 2000 条 / top_k 50 / 2 秒）、以及流式评测的定性结论（[A-10]）都指向相反的直觉。

6.3 尚未证明的问题

1. "确定性规则抽取 vs LLM 抽取"孰优，本次调研无法回答。 学术轨道专门为此标注了材料稀薄：没有任何论文做直接对比实验，粒度漂移也无人量化；最接近的是一条负面证据（抽取阶段会丢掉承载冲突的语境/时间/来源关系，见附录 B 的 TANGLE）。项目正在走的"确定性优先 + 语义兜底"路线，目前是设计选择而非有依据的选择。
2. "记忆单元数量增长 → 检索精度退化"没有一篇以此为主动实验的工作。 最接近的是 [A-10] 的定性结论与 [O-02] 的厂商阈值。项目自测的"案例 9→22 时噪声/查询 0.25→0.91"在外部没有可对标的曲线。
3. 拒答能力没有外部基线可对标。 它被写进公共口径（[A-12] 的 abstention、[A-11] 的选择性遗忘），却在公开报数中被系统性剔除（[A-14]、[I-07] 剔除对抗类；[O-01] 的公开记录只计 470/500 题）。这意味着项目前瞻测试里 3/3 失败的误报问题，既不能靠对标外部数字来定位，也不能用别人的分数

来安慰自己。

4. 被遗忘权/合规删除在学术侧无材料，在工业侧只有单侧证据（[I-05]）。删除是否真正传播到底层向量库/图库的副本与备份，三家厂商都没有说明（这是行业级空白，不是本次检索遗漏）。
5. "非向量架构在真实工作负载上会掉多少分"仍未定。唯一明确的反向证据（[A-05]：-22.0 / -26.5 个百分点）发生在 LongMemEval 级的历史规模与公开端点上，与项目场景相差数个数量级，它证明了"该测量"的必要性，而没有证明"该项目会掉分"。

6.4 若现在必须决策，我的建议

1. 维持"零向量库、零图库"的架构判断，并按 [O-01] 的形式把它从"立场"改写成可测的重访触发条件（建议直接采用其形状：规模阈值、p95 延迟阈值、以及"向量相比现有通道带来 +5–10% recall@5"这类效果阈值），写进 FRAMEWORK_READINESS.md 的重访清单。
2. 把 PENDING.md §2 的"通用词泄漏：保留不修"改判为"先修，且作为独立回归项"。理由是外部证据把精度治理的地位抬高了（[A-04]、[I-06]），而且项目自己的前瞻测试显示它是唯一已被证实的误报根因（NOVEL 3/3）。修法建议优先"跨通道去重"与"多组件键要求 ≥2 个组件命中"两条，因为它们在结构上等同于 [I-06] 的"先过滤后排序"。
3. 把下一个迭代的主要工程量投到写入侧元数据，按 [A-07] 的消融结论排序：先补通用生命周期字段（告警事件时间与案例入库时间分开存、来源、活跃/失效标志、槽位键），再考虑细分类型（细分类只买校准）；这一步同时兑现 [A-09] 的不可逆损失与 [I-06] 的溯源要求。
4. 把"补测 3：查询形态测试"按查询类型分层重做，而不是做一次整体自然语言召回测试（依据 [A-01]：同一底座在不同负载上一个通道可以完全不必要、在另一个负载上不可或缺）。
5. 补一项"拒答"验收测试：库外内容必须静默，且要以"硬负例 + 误报率"的形式写进指标（参考 [A-12] 的 abstention 与 [I-06] 的先过滤后排序）。当前 3/3 强档误报在外部证据里属于已被识别为结构性盲区的失败模式（[A-06]），不是可以靠调阈值消掉的瑕疵。
6. 图的问题按项目自己的路线图走，不必再等外部证据。[A-02]/[A-03]/[I-03] 已给出足够清晰的形态约束（稀疏三类关系 + 一跳 + 人工确认 + 不建全图 + 不做跨图遍历），足以支撑"条件采纳 B"落笔；剩余的不确定性在实现细节，不在方向。

6.5 建议继续跟踪的方向

- 装配/读取环节的公开标尺（[A-04] 所属切片的多跳公共天花板 ≤7%）是否会随新工作被打破——它直接决定"关系层能不能带来多跳收益"这个问题什么时候才有外部答案。
- 工作负载自适应的检索路由（[A-01] 类）是否会成为主流形态；若是，则"向量层的取舍"最终会被一个每查询决策的级联路由取代，而不是被一次架构选择决定。
- 记忆投毒的防御有效性度量：目前只有攻击侧数字（[A-06] 的效用曲线）与厂商的定性控制清单（[I-06]），没有防御侧的可比较结果（无 ASR 对照、无误报率）。这是本项目若要上关键生产环境必须自己补的一类证据。
- "删除是否传播到底层副本"是否会有厂商给出明确说明（当前三家都没有，[I-05]）。它决定托管记忆能否进入受监管场景。

7 局限

1. 检索截止 2026-09-28，此后发布的结果未纳入。本领域迭代极快（[I-04] 的模型栈在一年内即被更换），故本文的时效性衰减会比一般课题更快。
2. 渠道限制，且影响到具体结论：本机 github.com、raw.githubusercontent.com、cdn.jsdelivr.net、dl.acm.org、huggingface.co、api.openai.com 均不可达；shell 的 curl/git 因 TLS 与连接受限不可用，全部取数经 node fetch 与桥接通道完成。后果有三：开源轨道没有任何本地克隆，全部仓库证据经 api.github.com 获取，没有一条经过构建或运行；ACM/Elsevier 正文基本无法获取，非 arXiv 且无预印本版的同行评审来源可能存在系统性缺口；web_search 有 5 次因上游限流失败，学术轨道的发现主要依赖 arXiv API。
3. 部分条目只读到摘要或前半：学术轨道中 [A-08] 我只读到摘要与元数据（结果表未核）、[A-10] 只读到设计与方法章、[A-02] 的 66.6% 与"最高"的基线集合未核、[A-07] 的实验章未核（其数字来自摘要与引言）；这些条目内的数字均已注明来源层级。工业轨道中 [I-01]/[I-02]/[I-03]/[I-04]/[I-05] 等多条页面无绝对日期，无法支撑精确时效判断（已逐条标注）。开源轨道中 [O-05]/[O-06] 的许可未确认，多条未取 commit 哈希。我亲自复核了承重的四条（[A-14] 全文、[O-01] 的向量后端策略与基准记录、[O-02] 的融合公式与规模常量、[O-05] 的判分脚本），其余条目依赖子代理账本 + 我对其原文的抽读；凡我未亲自打开的一手页面，其数字一律标注为"来源自陈"。
4. 四维评分含主观权重，调整权重会改变排序。对权重最敏感的是 [A-05]（若把置信度权重提高，它会因单作者与私有主基准而下滑；若把意义权重提高，它会因"唯一反向证据"进一步上升）与 [A-13]/[A-14]（时效权重再压低会使其跌出榜单）。但本文的主要结论不依赖这两条中的任何一条的排名。
5. 本报告没有重跑项目自身的任何脚本，所有关于本项目现状的数字（22 案例、95 查询、231 对 gold、C1–C5、噪声 0.25→0.91、NOVEL 3/3 等）均引自项目文档与评测报告，未复算。
6. 未纳入本报告的相关工作及原因：通用 RAG 语料投毒一族（属排除项）、纯向量库跑分横评（属排除项）、模型侧长上下文与位置编码研究（属排除项）、以及长尾厂商营销页（多为二手或无日期）。少数已读到但排在 30 条之外的值得注意项见附录 B。
7. 一处未核验的外部线索：第三方对 LoCoMo 答案键的审计（称 1,540 题中 6.4% 答案键有错、LLM judge 对"话题对但细节全错"的答案放行率最高约 63%）只在检索结果中出现，本次未打开原文，故本报告只在 [A-13] 处把它作为"存在效度质疑"的背景提及，未采用其具体数字。
8. PDF 的核验方式：本次依赖渲染器自检（内容块 511/511 全部出现在 PDF 抽取文本中、0 溢出、0 碰撞、字体已嵌入、生成的目录 28/28 条页码正确），并已按其提示修掉两处字体不支持的字符（上标 g 与 U+2212 减号）后重渲。本环境当前模型不声明图像输入、视觉桥接亦未配置，因此未做人眼级的版式复核；页面预览图已生成在 调研报告-Agent记忆产品落地-20260928.preview/（page-01、page-02），可直接打开查看。

附录 A 评分明细

时效按本课题节奏校准（2026 年产出 10、2025 年 8、2024 年 6 起）。总分 = 0.30×贡献 + 0.25×意义 + 0.20×时效 + 0.25×置信度。

编号	结果	轨道	贡献	意义	时效	置信度	总分	引用位置
O-01	akitaonrails/ai-memory	开源	9.0	9.0	9.5	8.0	8.85	§4、§6.4
I-01	Letta MemFS	工业	8.5	9.0	10.0	8.5	8.93	§3.1

编号	结果	轨道	贡献	意义	时效	置信度	总分	引用位置
I-02	Claude Code memory	工业	8.5	9.0	9.0	8.5	8.73	§3.1
A-11	MemoryAgentBench	学术	8.5	9.0	9.0	8.0	8.60	§2.4
O-02	mem0ai/mem0（代码）	开源	8.0	8.5	9.5	8.5	8.55	§4、§5
A-05	LSREP / ICE v2	学术	9.0	9.0	10.0	6.0	8.45	§2.3、§6.1
A-12	LongMemEval	学术	8.0	9.0	8.0	8.5	8.38	§2.4、§6.3
A-06	Utility Under Attack	学术	9.0	8.5	10.0	6.0	8.33	§2.3、§6.4
I-03	Zep Konig（自建图库）	工业	8.5	8.5	10.0	6.5	8.30	§3.2、§5
I-10	OWASP MemoryTrap / ASI06	工业	8.0	8.5	9.5	7.5	8.30	§3.5
O-03	letta → letta-code	开源	8.0	8.5	9.5	7.5	8.30	§4
I-05	Anthropic 记忆保留与删除	工业	8.0	8.5	8.0	8.5	8.25	§3.3、§6.3
A-03	Grounded Continuation	学术	9.0	8.0	10.0	6.0	8.20	§2.2、§6.4
A-04	Reliable Post-Retrieval Assembly	学术	8.5	8.0	10.0	6.5	8.18	§2.2、§6.1
A-07	Fortunate Recall	学术	8.5	8.5	10.0	6.0	8.18	§2.3、§6.4
O-04	getzep/graphiti	开源	7.5	8.0	9.5	8.0	8.15	§4、§5
I-06	Zep 记忆投毒防御	工业	8.0	8.5	10.0	6.5	8.15	§3.5、§5
A-09	What Eviction Destroys	学术	8.5	8.5	10.0	5.5	8.05	§2.3、§6.1
A-02	MemoryLACE	学术	8.5	8.0	10.0	5.0	7.80	§2.2、§6.1
A-08	Graph Memory: At What Cost	学术	8.0	8.0	10.0	5.5	7.78	§2.3、§6.1
O-06	LongMemEval 评测脚本	开源	7.5	8.0	8.5	7.0	7.70	§4、§5
I-09	AWS AgentCore 定价	工业	7.0	7.5	8.5	8.0	7.68	§3.5、§5
A-01	AgentIR	学术	8.0	8.0	10.0	5.0	7.65	§2.1、§6.4
O-05	LoCoMo 评测脚本	开源	7.5	8.0	7.0	7.5	7.53	§4、§5
I-04	Zep Research 基准页	工业	7.5	8.0	8.0	6.0	7.35	§3.4、§5
A-10	Neuromem	学术	7.5	8.0	9.0	5.0	7.30	§2.3、§6.3
I-07	Zep 对 Mem0 的质疑	工业	8.0	8.5	7.0	5.0	7.18	§3.4、§5
I-08	Mem0 对 Zep 的复算质疑	工业	8.0	8.0	7.0	5.0	7.05	§3.4、§5
A-13	LoCoMo	学术	7.0	8.0	6.0	7.0	7.05	§2.4、§5
A-14	Mem0 论文	学术	7.0	9.0	7.0	4.0	6.75	§2.4、§6.2

组织集中度检查：涉及 Mem0 的条目 3 条（[A-14]、[I-08]、[O-02]）占 10%；涉及 Zep 的条目 5 条（[I-03]、[I-04]、[I-06]、[I-07]、[O-04]）占 17%；均低于三分之一上限。

附录 B 未纳入但值得注意

以下条目在读到时被评为相关性边缘、置信度偏低、或与已选结果重复，故未进入 30 条正文（**已读到的内容与访问深度记录在 `evidence/` 下的三份子代理账本中**）。

- TANGLE（arXiv:2608.13921）：541 例"真正无法消解"的记忆冲突基准，明确把"抽取阶段丢掉承载冲突的语境/时间/来源关系"指为上游瓶颈——与本项目"人工确认边"的设计直接相关，因摘要无任何数字而未纳入。
- Selection Integrity for LLM Graph Memory（arXiv:2606.12290）：论证一旦关系边可由非信任来源自动写入，"选择"本身成为攻击面，且基于记录来源的防御按构造失明（被引用的证据全部干净）。这是"人工确认 vs 全自动建边"在安全侧的量化理由，因我未读到其判定与实验章节而列为旁证。
- PMPA（arXiv:2609.13889）与 MPBench（arXiv:2606.04329）：经记忆持久化的注入在真实 harness 上的成功率（OpenClaw 73.7%、Claude Code 66.9%；跨会话 55.5% / 81.7%）与"四条写入通道 / 九类结构性脆弱点"的分类法。与 [A-06]、[I-10] 同向，未纳入以控制条数。
- EnSIMem（arXiv:2609.27279）：[实体][类型][属性:值] → episode_id 索引 + 原文保留，并把"属性粒度"提为核心折衷——与本项目正在讨论的"动作/实体粒度"问题几乎同一命题；因摘要无数字、实验章未读而未纳入。
- 两份综述：arXiv:2602.06052《A Survey of Agent Memory in the Second Half》（三维视角：记忆基质 / 认知机制 / 记忆主体，明确把领域定位为"从跑分转向严格真实世界评测"）与 arXiv:2605.06716《From Storage to Experience》（存储 → 反思 → 经验三阶段）。综述不产生新证据，仅作分类坐标。
- LongMemEval-V2（arXiv:2605.12493）：451 题、轨迹最多 500 条 / 115M tokens，最佳方法为"把轨迹存文件 + 调用编码 agent 取证"（72.5%，优于最强 RAG 基线 48.5%），代价是高延迟。它代表 2026 年的前沿方向（agentic retrieval），与本项目的符号检索是可组合关系而非替代关系。
- 记忆投毒一族（arXiv:2601.05504、2603.20357、2609.00523 等）：本次经 arXiv API 检出多篇 2026 年工作，未逐篇阅读，故不描述其结论。
- 工业侧的旁证来源（均为一手页面，已存档于 evidence/pages/，未单列条目）：Mem0 定价页与 Graph Memory 文档（图记忆仅 $249/月 Pro 档提供、图只做共现边与排序提升）、Mem0 "How it works"（四信号检索含 BM25；OSS 形态无图记忆）、Zep 定价页（检索/存储/图存储 0 credit、只按写入字节计费、600 RPM）、Pinecone 定价页（读单价是写单价的 3–6 倍、重排序单列计价 $2/千次）、OWASP LLM08:2025、Google Agent Platform Memory Bank（identity 作用域隔离、TTL、memory revisions、跨边界污染防护）、LangChain Memory（按 key 直取 / 内容 filter / 向量排序三种并列原语）、AWS AgentCore Memory devguide、Mem0 FAQs（账号删除为异步分钟级）。
- 开源侧的旁证项目（已读元数据或部分代码）：mem0ai/memory-benchmarks（厂商自建 harness，判分含"部分正确"/"复述也算对"等宽松规则，oss 与 platform 结果分开存放）、HUST-AI-HYZ/MemoryAgentBench（其判分提示词与 LongMemEval 逐字相同，故两者不是独立证据）、agiresearch/A-mem（依赖 ChromaDB + rank_bm25 + sentence-transformers，最后推送 2025-12）、topoteretes/cognee（图/向量/关系/缓存四类适配器并存，默认后端未确认）、BAI-LAB/MemoryOS（EMNLP 2025 Oral，仅元数据）、n-r-w/knowledgegraph-mcp（26 stars，仅元数据，不应据其宣传语推断可用性）。
- 未获取、不得引用其内容：MemSecBench（alphaxiv.org/abs/2607.27080，仅见检索片段）；agiresearch/OpenAI-Memory（GitHub API 返回 404，该 owner/repo 组合不存在，若需引用须另找准确仓库名）；AWS AgentCore 配额页（4 个候选 URL 均失败）、Zep Graphiti 文档页、OWASP ASI06 条目正文、Mem0 速率限制（厂商未公开）——这四项构成产品化约束上明确的核验缺口。
- 未纳入的其他候选：MemConflict（arXiv:2605.20926）、TWIST（arXiv:2609.28575）、The Price of Safety（arXiv:2609.22818）、StateFuse（arXiv:2607.05844）、MRAgent（arXiv:2606.06036）、MemGPT（2310.08560）与 Generative Agents（2304.03442）、Locomo-Plus（2602.10715）、BEAM（2510.27246）、以及第三方 LoCoMo 答案键审计的三条线索（均只见检索片段、未打开）。

附录 C 键位对照

正文的 A/I/O 键是本报告重新编号的结果，与 `evidence/` 下三份子代理账本的原始键不同。为便于复核，对照如下（本代理自采条目见 `evidence/lead-agent.md`，其临时键为 `L-*`）。

报告键	子代理账本原键（文件）	备注
A-01	无（主代理自采）	AgentIR，主代理读摘要；见 lead-agent.md 未核验线索第 5 项
A-02	账本 A-08（academic.md）	MemoryLACE
A-03	账本 A-14（academic.md）	Grounded Continuation
A-04	账本 A-15（academic.md）	Reliable Post-Retrieval Assembly
A-05	账本 A-06（academic.md）	LSREP / ICE v2
A-06	账本 A-16（academic.md）	Utility Under Attack
A-07	账本 A-12（academic.md）	Fortunate Recall
A-08	账本 A-07（academic.md）	Graph Memory: At What Cost
A-09	账本 A-05（academic.md）	What Eviction Destroys
A-10	账本 A-19（academic.md）	Neuromem
A-11	账本 A-01（academic.md）	MemoryAgentBench
A-12	账本 A-02（academic.md）+ 主代理 L-A-02	LongMemEval（主编补读摘要）
A-13	账本 A-03（academic.md）+ 主代理 L-A-04	LoCoMo（主编补读摘要）
A-14	账本 A-10（academic.md）+ 主代理 L-A-01	Mem0 论文（主代理逐节核读全文）
I-01	账本 I-15（industry.md）	Letta MemFS
I-02	账本 I-09（industry.md）	Claude Code memory
I-03	账本 I-13（industry.md）	Zep Konig
I-04	账本 I-12（industry.md）	Zep Research
I-05	账本 I-08（industry.md）	Anthropic 记忆保留与删除
I-06	账本 I-14（industry.md）	Zep 投毒防御
I-07	主代理 L-I-01（lead-agent.md）	Zep 反驳博文（主代理读整页）
I-08	主代理 L-I-02（lead-agent.md）	getzep/zep-papers issue #5（主代理读完整正文）
I-09	账本 I-01（industry.md）	AWS AgentCore 定价
I-10	账本 I-19 / I-20（industry.md）	OWASP ASI06 与 MemoryTrap
O-01	账本 O-04（opensource.md）	ai-memory（主代理亲验策略文档与基准记录）
O-02	账本 O-02（opensource.md）	mem0（主代理亲验 scoring.py 与 notices.py）
O-03	账本 O-03（opensource.md）	letta → letta-code
O-04	账本 O-01（opensource.md）	graphiti
O-05	账本 O-05（opensource.md）	LoCoMo 评测脚本（主代理亲验 293 行判分逻辑）
O-06	账本 O-06（opensource.md）	LongMemEval 评测脚本